

Sistemas de Operação

Processor scheduling

Artur Pereira <artur@ua.pt>

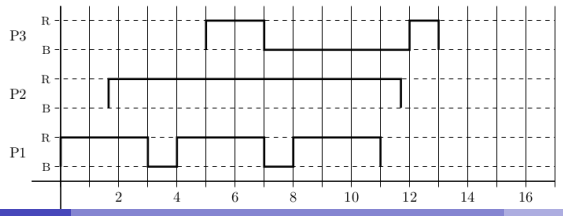
DETI / Universidade de Aveiro

Outline

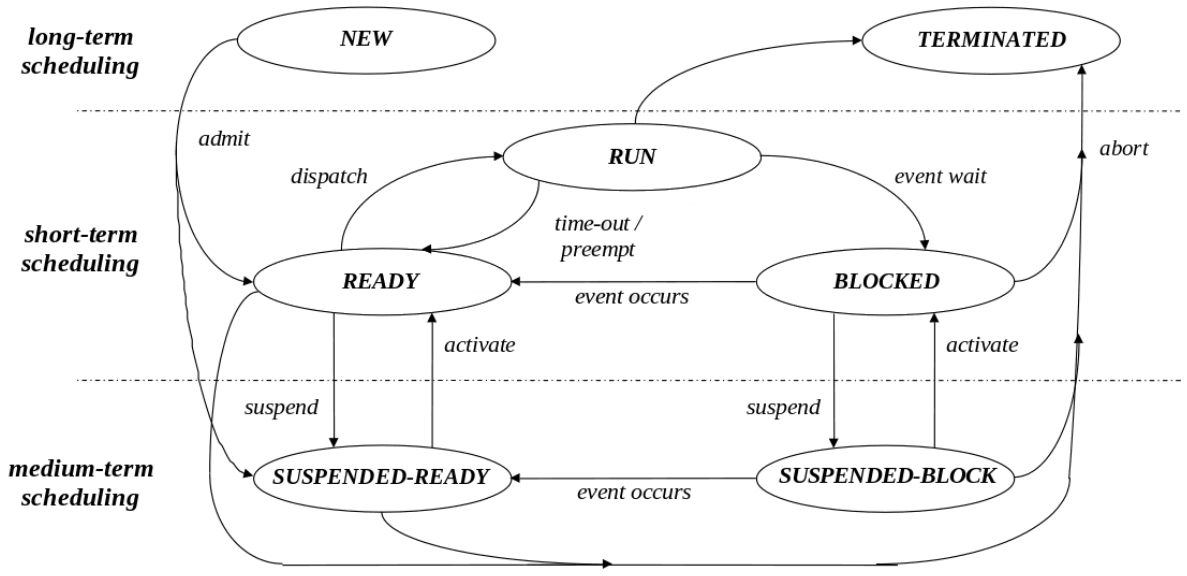
1. Introduction
2. Assessing a (short-term) scheduling algorithm
3. Priorities
4. Scheduling algorithms
5. Scheduling in Linux
6. Bibliography

Processor scheduler
Definition

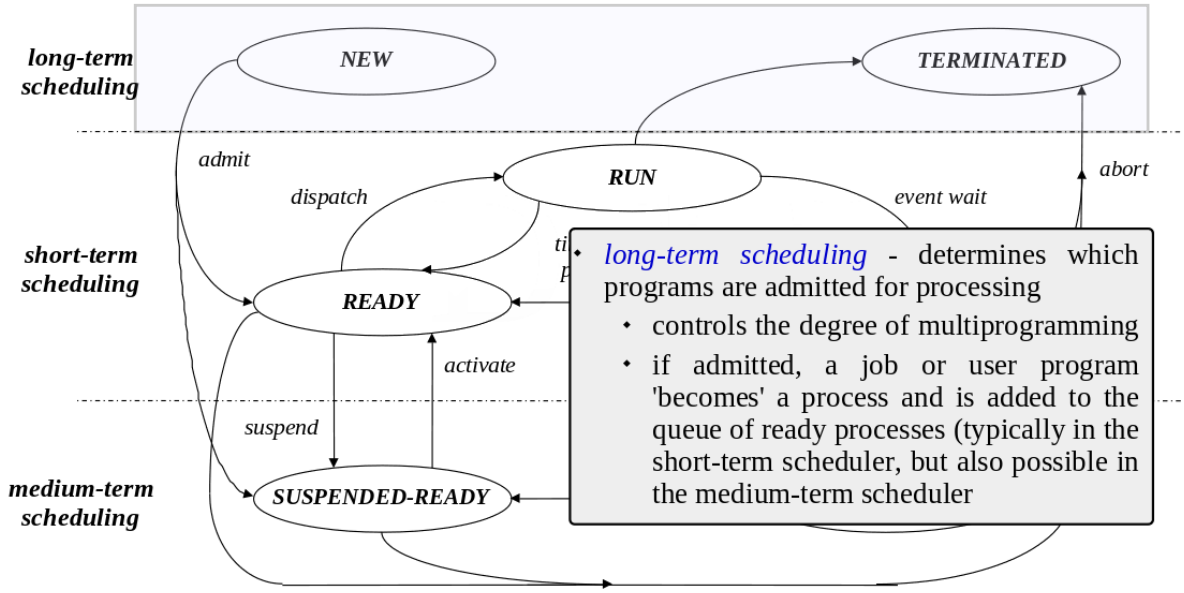
- When seen by its own, the execution of a process is an alternate sequence of two type of periods:
  - CPU burst – executing CPU instructions
  - I/O burst – waiting for the completion of an I/O request
- Based on this, a process can be classified as:
  - I/O-bound – if it has many short CPU bursts
  - CPU-bound (or processor-bound) – if it has (few) long CPU bursts
- The idea behind multiprogramming is to take advantage of the I/O burst periods to put other processes using the CPU
- The processor scheduler is the component of the operating system responsible for decising how the CPU is assigned for the execution of the CPU bursts of the several processes that coexist in the system



Processor scheduler
Levels in processor scheduling

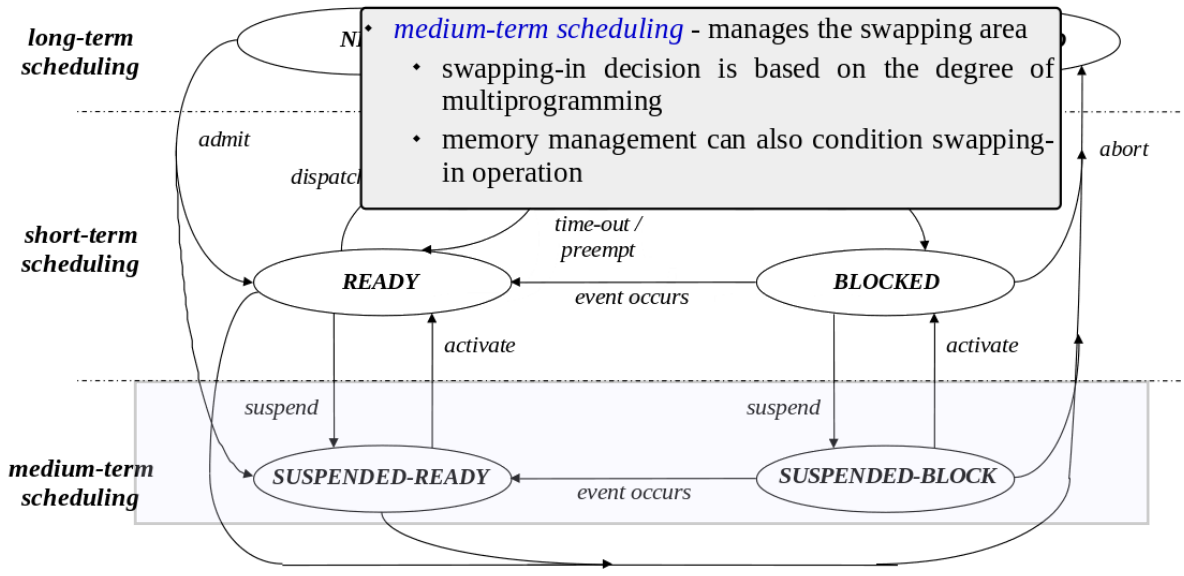


Processor scheduler
Long-term scheduling



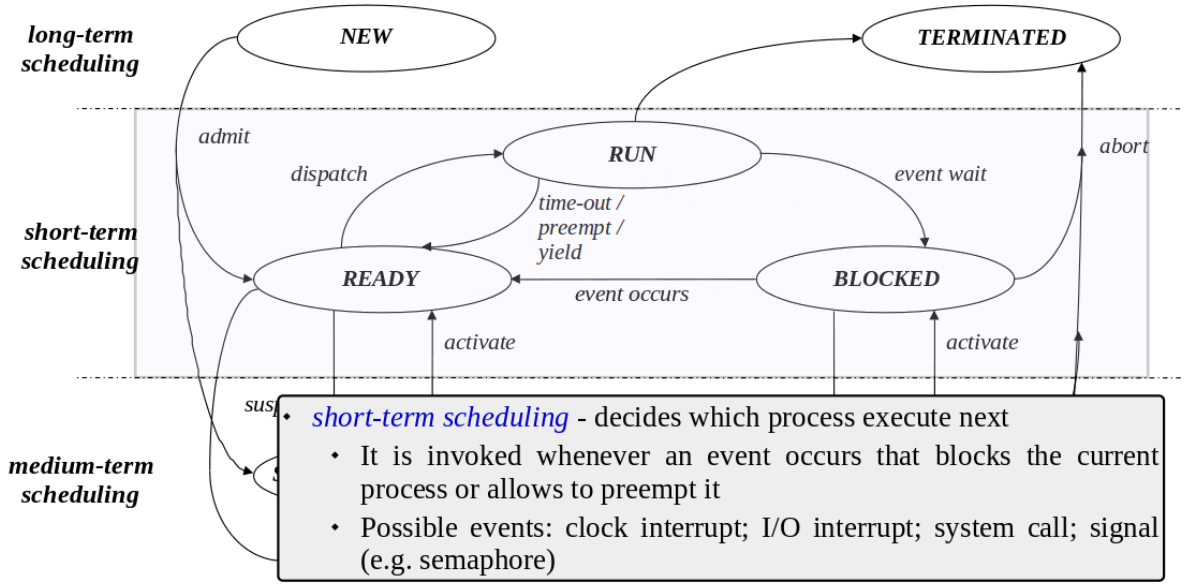
- *long-term scheduling* - determines which programs are admitted for processing
  - controls the degree of multiprogramming
  - if admitted, a job or user program 'becomes' a process and is added to the queue of ready processes (typically in the short-term scheduler, but also possible in the medium-term scheduler

Processor scheduler
Medium-term scheduling



- *medium-term scheduling* - manages the swapping area
  - swapping-in decision is based on the degree of multiprogramming
  - memory management can also condition swapping-in operation

Processor scheduler
Short-term scheduling



- *short-term scheduling* - decides which process execute next
  - It is invoked whenever an event occurs that blocks the current process or allows to preempt it
  - Possible events: clock interrupt; I/O interrupt; system call; signal (e.g. semaphore)

Short-term processor scheduler
Preemption & non-preemption

- The short-term processor scheduler can be preemptive or non-preemptive
- Non-preemptive scheduling – a process keeps the processor until it blocks, ends, or yields it
  - Transitions time-out and preempt do not exist
  - Typical in batch systems. *Why?*
- Preemptive scheduling – a process can lose the processor due to external reasons
  - by exhaustion of the assigned time quantum (time-out transition)
  - because a more priority process becomes ready to run (preempt transition)
- Interactive systems must be preemptive. *Why?*
- What type to use in a real-time system? Why?

Scheduling algorithms
Evaluation criteria

- The main objective of short-term scheduling is to allocate processor time in order to optimize some objective function of the system behavior
- A set of criteria must be established for the evaluation
- These criteria can be seen from two different perpectives:
  - User-oriented criteria – related to the behavior of the system as perceived by the individual user or process
  - System-oriented criteria – related to the effective and efficient utilization of the processor
  - A criterion can be a direct/indirect quantitative measure or just a qualitative evaluation
- Scheduling criteria are interdependent, thus it is impossible to optimize all of them simultaneously

Scheduling criteria
User-oriented scheduling criteria

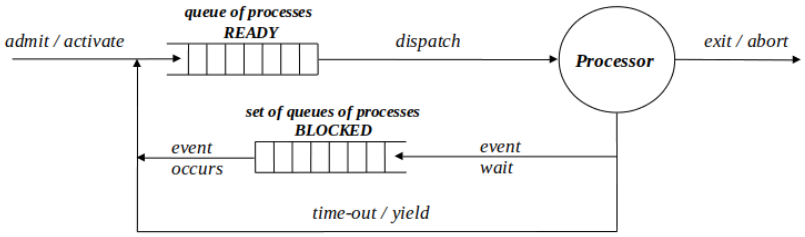
- Turnaround time – interval of time between the submission of a process/job and its completion (includes actual execution time plus time spent waiting for resources, including the processor)
  - appropriate measure for a batch job
  - should be minimized
- Waiting time – sum of periods spent by a process waiting in the ready state
  - should be minimized
- Response time – time from the submission of a request until the response begins to be received
  - appropriate measure for an interactive process
  - should be minimized
  - but also the number of interactive processes with acceptable response time should also be maximized
- Deadlines – time of completion of a process
  - percentage of deadlines met should be maximized, even subordinating other goals
- Predictability – how response is affected by the load on the system
  - A given job should run in about the same amount of time and at about the same cost regardless of the load on the system

Scheduling criteria
System-oriented scheduling criteria

- Fairness – equality of treatment
  - In the absence of guidance, processes should be treated the same, and no process should suffer starvation
- Throughput – number of processes completed per unit of time
  - measures the amount of work being performed by the system
  - should be maximized
  - depends on the average lengths of processes but also on the scheduling policy
- Processor utilization – percentage of time that the processor is busy
  - should be maximized (specially in expensive shared systems)
- Enforcing priorities
  - higher-priority processes should be favoured

- As referred to before, it is impossible to satisfy all criteria simultaneously
- Those to favour depends on application

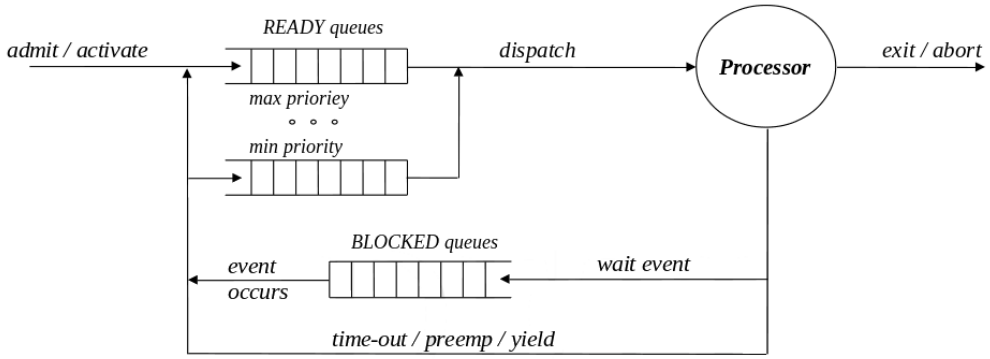
Priorities
Without priorities, favouring fearness



- All processes are equal, being served in order of arrival to the READY queue
  - In non-preemptive scheduling, it is normally referred to as first-come, first served (FCFS) – the time-out transition does not exist [a]
  - In preemptive scheduling, it is normally referred to as round robin – the time-out transition exist
- Easy to implement
- Favours CPU-bound processes over I/O-bound ones
- In interactive systems, the time quantum should be carefully chosen in order to get a good compromise between fairness and response time

[a] Also the preempt transition does not exist, since it is assumed there are no priorities

Priorities
Enforcing priorities



- Often, being all the same is not the most appropriate
  - in interactive systems, minimizing response time requires I/O-bound processes to be privileged
  - in real-time systems, there are processes (for example, those associated with alarm events or operational actions) that have severe time constraints
- To address this, processes can be grouped in different levels of priority
  - higher-priority processes are selected first for execution
  - lower-priority processes can suffer starvation

Priorities
Types of priorities

- Priorities can be:
  - static – if they do not change over time
  - dynamic – if they depend on the past history of execution of the processes

- Static priorities:
  - Processes are grouped into fixed priority classes, according to their relative importance
  - Clear risk of starvation of lower-priority processes
  - The most unfair discipline
  - Typical in real-time systems – why?

- Dynamic, deterministically changing priorities:
  - When a process is created, a given priority level is assigned to it
  - on time-out the priority is decremented
  - on event wait the priority is incremented
  - when a minimum value is reached, the priority is set to the initial value

Priorities
Types of priorities (2)

- Dynamic priorities:
  - Priority classes are functionally defined
  - In interactive systems, change of class can be based on how the last execution window was used
    - level 1 (highest priority): terminals – a process enters this class on event occurs if it was waiting for data from the standard input device
    - level 2: generic I/O – a process enters this class on event occurs if it was waiting for data from another type of input device
    - level 3: small time quantum – a process enters this class on time-out
    - level 4: (lowest priority): large time quantum – a process enters this class after a successive number of time-outs.
      They are clearly CPU-bound processes and the ideia is given them large execution windows, less times

Priorities
Types of priorities (3)

- Dynamic priorities:
  - In batch systems, the turnaround time should be minimized
  - If estimates of the execution times of a set of processes are known in advance, it is possible to establish an order for the execution of the processes that minimizes the average turnaround time of the group.
  - Assume N jobs are submitted at time 0, whose estimates of the execution times are t_{e_n}, with $n = 1, 2, \cdots, N$.
    - The turnaround time of job i is given by

$$t_{t_i} = t_{e_1} + t_{e_2} + \cdots + t_{e_i}$$

    - The average turnaround time is given by

$$t_m = \frac{1}{N}\sum_{i=1}^{N} t_{t_i} = t_{e_1} + \frac{N-1}{N} t_{e_2} + \cdots + \frac{1}{N} t_{e_N}$$

    - t_m is minimum if jobs are scheduled in ascending order of the estimated execution times

Priorities
Types of priorities (4)

- Dynamic priorities:
  - An approach similar to the previous one can be used in interactive systems
  - The idea is to estimate the occupancy fraction of the next execution window, based on the occupation of the past windows, and assign the processor to the process for which this estimate is the lowest
  - Let e_1 be the estimate of the occupancy fraction of the first execution window assigned to a process and let f_1 be the first fraction effectively verified. Then:
    - estimate e_2 is given by

$$e_2 = \alpha.e_1 + (1-\alpha).f_1 \quad , \quad \alpha \in [0,1]$$

    - estimate e_N is given by

$$\begin{aligned} e_N &= \alpha.e_{N-1} + (1-\alpha).f_{N-1} \quad , \quad \alpha \in [0,1] \\ &= \alpha^{N-1}.e_1 + \alpha^{N-2}.(1-\alpha).f_1 + \cdots + \alpha.(1-\alpha).f_{N-2} + (1-\alpha).f_{N-1} \end{aligned}$$

  - coefficient α is used to control how much the past history of execution influences the present estimate

Priorities
Types of priorities (5)

- In the previous approach, CPU-bound processes can suffer starvation
- To overcome that problem, the aging of a process in the READY queue can be part of the equation
- Let R be such time, typically normalized in terms of the duration of the execution interval.
  - Then, priority p of a process can be given by

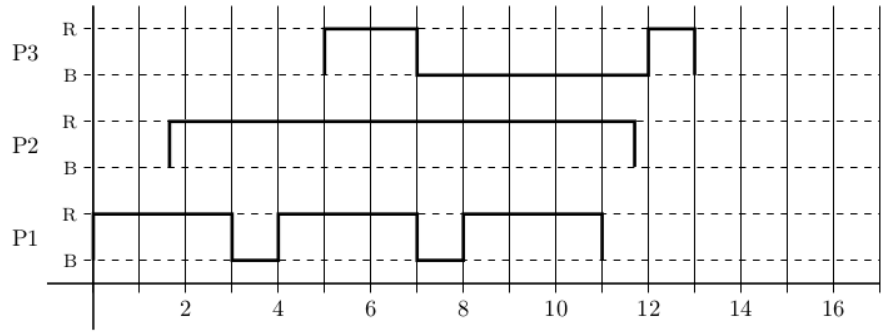
$$p = \frac{1 + \beta.R}{e_N}$$

  where β is a coefficient that controls how much the aging weights in the priority

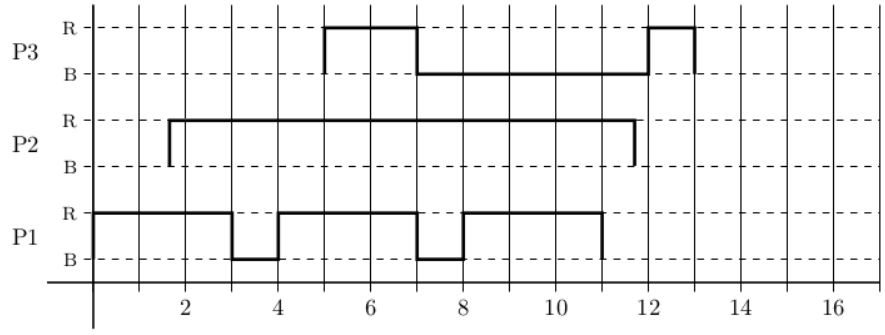
Scheduling policies
FCFS

- First-Come-First-Server (FCFS) scheduler
  - Also known as First-In-First-Out (FIFO)
  - The oldest process in the READY queue is the first to be selected
  - Non-preemptive (in strict sense)
    - Can be combined with a priority schema (in which case preemption could exist)
  - Favours CPU-bound processes over I/O-bound

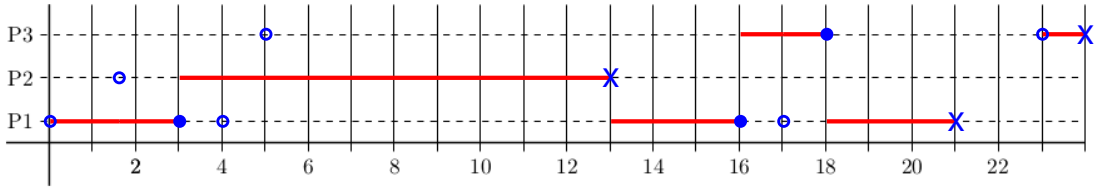
- Let's look at an example



Scheduling policies
FCFS – example



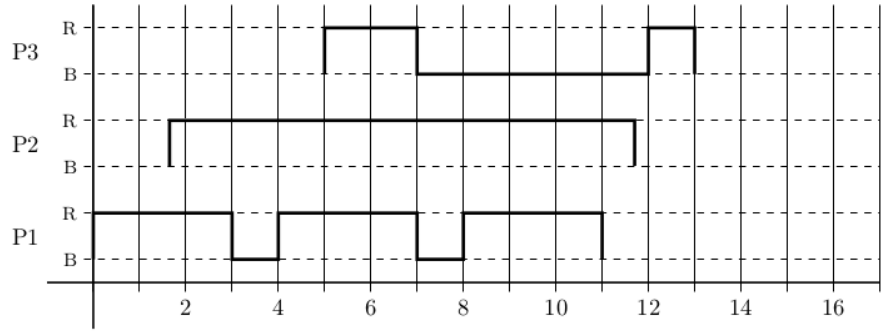
- Draw processor utilization, assuming FCFS policy and no priorities



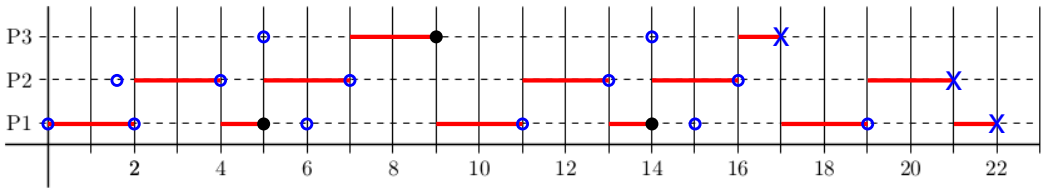
Scheduling policies
RR

- Round robin (RR) scheduler
  - Preemptive (base on a clock)
    - Each process is given a maximum slice of time before being preempted (time quantum)
    - Also known as time slicing
  - The oldest process in the READY queue is the first one to be selected (no priorities)
    - Can be combined with a priority schema
  - The principal design issue is the time quantum
    - very short is good, because short processes will move through the system quickly and response time is minimized
    - very short is bad, because every context switching involves a processing overhead
  - Effective in general purpose time-sharing systems and in transaction processing systems
  - Favours CPU-bound processes over I/O-bound

Scheduling policies
RR – example



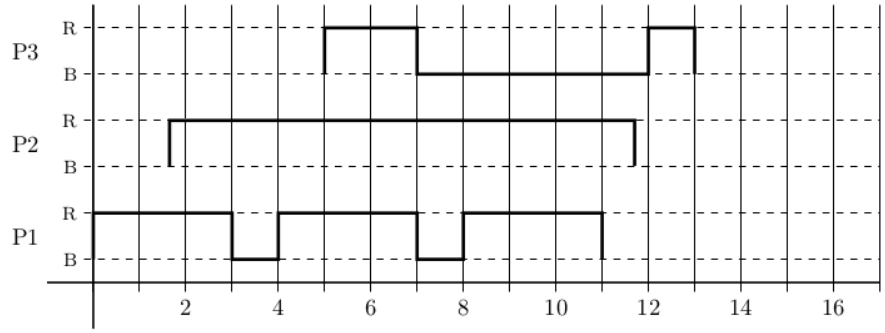
- Draw processor utilization, assuming RR policy with a time quantum of 2 and no priorities



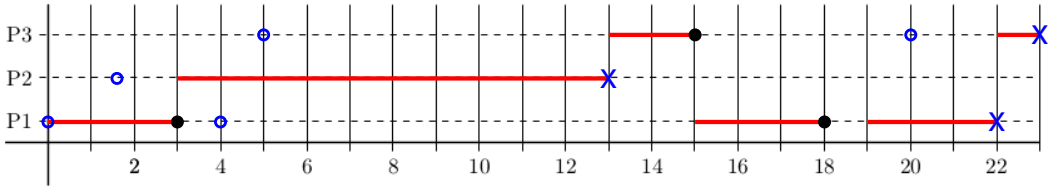
Scheduling policies
SPN

- Shortest Process Next (SPN) scheduler
  - Also known as shortest job first (SJF)
  - Non-preemptive
  - The process with the shortest expected next CPU burst time is selected next
    - FCFS is used to tie up (in case several processes have the same burst time)
  - Maximum throughput
  - Minimum average waiting time and turnaround time
  - Risk of starvation for longer processes
  - Requires the knowledge in advance of the (expected) (CPU-burst) processing time
  - Used in long-term scheduling in batch systems
    - where users are motivated to estimate the process time limit accurately

Scheduling policies
SPN – example



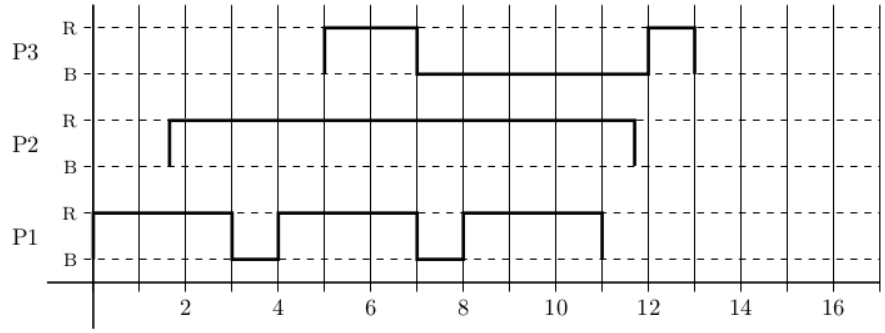
- Draw processor utilization, assuming SPN policy and no priorities



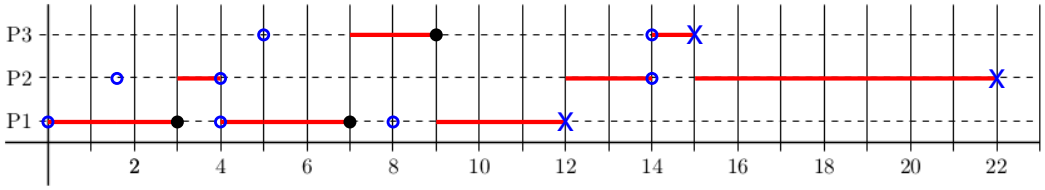
Scheduling policies
SRT

- Shortest Remaining Time (SRT) scheduler
  - Preemptive at arrival (of a process to the READY queue)
  - Preemptive version of SPN
  - The process with the shortest expected remaining CPU-burst time is selected next
    - FCFS is used to tie up (in case several processes have the same burst time)
  - Maximum throughput
  - Minimum average waiting time and turnaround time
  - Risk of starvation for longer processes
  - Requires the knowledge in advance of the (expected) (CPU-burst) processing time

Scheduling policies
SRT – example



- Draw processor utilization, assuming SRT policy and no static priorities



Scheduling policies in Linux
Different classes

- Linux (2.4 and earlier) considers 3 scheduling classes, each with multiple priority levels:
  - SCHED_FIFO – FIFO real-time threads, with priorities
    - a running thread in this class is preempted only if a higher priority process (of the same class) becomes ready
    - a running thread can voluntarily give up the processor, executing primitive `sched_yield`
    - within the same priority an FCFS discipline is used
  - SCHED_RR – Round-robin real-time threads, with priorities
    - additionally to previous class, a running process in this class is preempted if its time quantum ends
  - SCHED_OTHER – non-real-time threads
    - can only execute if no real-time thread is ready to execute
    - associated to user processes
    - the scheduling police changed along kernel versions
- priorities range from 0 to 99 for real-time threads and 100 to 139 for the others
- `nice` system call allows to change the priority of non-real time threads

Scheduling policies in Linux
Traditional algorithm for the `SCHED_OTHER`

- In class SCHED_OTHER, priorities are based on credits
- Credits of the running process are decremented at every RTC interrupt
  - the process is preempted when zero credits are reached
- When all ready processes have zero credits, new credits for all processes, including those that are blocked, are calculated
  - Recalculation is done based on formula

$$\text{CPU}(i) = \frac{\text{CPU}(i-1)}{2} + \text{PBase} + \text{nice}$$

  - Past history of execution and priorities are combined in the algorithm
- Response time of I/O-bound processes is minimized
- Starvation of processor-bound processes is avoided
- Not adequate for multiple processors and bad if the number of processes is high
  - it uses a single runqueue for all processors in an SMP system

Scheduling policies in Linux
New (?) algorithm for the SCHED_OTHER

- From version 2.6.23, Linux started using a scheduling algorithm for the SCHED_OTHER (SCHED_NORMAL) policy class known as completely fair scheduler (CFS)
- Schedule is based on a virtual run time value (`vruntime`), which records how long a thread has run so far
  - the virtual run time value is related to the physical run time and the priority of the thread
  - higher priorities shrinks the physical run time
- The scheduler selects for execution the thread with the smallest virtual run time value
  - a higher-priority thread that becomes ready to run can preempt a lower-priority thread
  - thus, I/O-bound threads may preempt CPU-bound ones
- The Linux CFS scheduler provides an efficient algorithm for selecting which thread to run next, based on a red-black tree;

Bibliography

- Operating Systems: Internals and Design Principles, W. Stallings, Prentice-Hall International Editions, 7th Ed, 2012
  - Chapter 9: Uniprocessor Scheduling (sections 9.1 to 9.3)
  - Chapter 10: Multiprocessor and Real-Time Scheduling (section 10.3)
- Operating Systems Concepts, A. Silberschatz, P. Galvin and G. Gagne, John Wiley & Sons, 9th Ed, 2013
  - Chapter 6: Process scheduling (sections 6.1 to 6.3 and 6.7.1)
- Modern Operating Systems, A. Tanenbaum and H. Bos, Pearson Education Limited, 4th Ed, 2015
  - Chapter 2: Processes and Threads (section 2.4)